SECTION 1

From Biological Neurons to Backpropagation

- **Historical arc:** McCulloch–Pitts units, Perceptrons, AI winters, and the 1986 backpropagation revival enabled training deep stacks.
- **Threshold logic units:** Weighted sums plus step functions implement linear classifiers; multi-output Perceptrons use dense weight matrices.
- **Perceptron limits:** Single-layer models cannot solve nonlinear problems such as XOR; Minsky and Papert highlighted these gaps.
- **Multilayer perceptrons:** Hidden layers with nonlinear activations compose nonlinear decision regions in feedforward networks.
- **Backpropagation:** Forward pass computes outputs; reverse-mode autodiff propagates loss gradients to every weight and bias; SGD updates parameters.
- **Activations:** Sigmoid and tanh are smooth for gradients; ReLU is the default hidden activation for speed and practice; stacks without nonlinearity collapse to one linear map.

SECTION 2

Regression and Classification MLPs

- **Regression heads:** One neuron per target; output activations optional (ReLU/softplus for positivity, tanh/logistic for bounded ranges); losses include MSE, MAE, or Huber for outliers.
- **Binary classification:** Single logistic output interprets positive-class probability.
- **Multilabel:** Multiple sigmoid outputs allow independent labels without summing to one.
- **Multiclass:** Softmax over one neuron per class with cross-entropy when classes are mutually exclusive.

SECTION 3

Implementing MLPs with the Sequential API

- **tf.keras:** TensorFlow's bundled Keras API is used for examples; models compile, fit, evaluate, and predict like Scikit-Learn estimators.
- **Fashion MNIST:** Built-in loader, validation split, pixel scaling to [0,1]`, `Flatten`, hidden Dense+ReLU, softmax** output.
- **Training workflow:** `compile` sets optimizer and loss; `fit` tracks `history`; `evaluate` and `predict` score held-out data.
- **California housing regression:** Sequential MLP with MSE loss demonstrates vector regression with the same API pattern.

SECTION 4

Functional API for Complex Architectures

- **Layer graph:** `Input` objects and layer calls `layer(tensor)` define DAGs beyond a single stack.
- **Wide & Deep:** Concatenate raw inputs with deep pathway outputs before the final Dense layer.
- **Multiple inputs and outputs:** Lists or dicts feed `Model(inputs=..., outputs=...)`; `fit` accepts tuple inputs and label lists with per-output losses and weights.
- **Auxiliary heads:** Extra outputs act as regularizers, weighted lower than the primary loss during training.

SECTION 5

Subclassing API

- **`keras.Model` subclasses:** Layers created in `__init__`; `call` defines forward computation with imperative logic.
- **Dynamic behavior:** Supports loops, branches, and custom TensorFlow ops for research prototypes.
- **Trade-offs:** Harder to serialize, clone, or introspect than declarative models; `summary` shows layers but not connectivity; weights can still be saved with `save_weights`.

SECTION 6

Saving Models, Callbacks, and TensorBoard

- **`model.save` / `load_model`:** HDF5 stores architecture, weights, and optimizer for Sequential and Functional models; subclassed models rely mainly on weight checkpoints.
- **Callbacks:** `ModelCheckpoint` stores periodic or best validation models; `EarlyStopping` halts training with `restore_best_weights`.
- **Custom callbacks:** Hook epoch and batch events for logging or diagnostics.
- **TensorBoard:** `TensorBoard` callback writes event files per run; the server visualizes scalars, graphs, and profiles; low-level `tf.summary` can log extra tensors, images, and audio.